LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
SEMESTER III



TUGAS PRAKTIKUM ACARA 27

NAMA:

RAJATA PRINGGIT BINTANG BIHAR

GOLONGAN:

A

NIM:

E41250264

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Pada Acara 25 dan 26, dashboard admin sudah dilindungi autentikasi dan pembatasan peran, serta mampu menyimpan foto mahasiswa. Penambahan dan perubahan data masih dilakukan lewat dialog (*dialog*) di dalam halaman dashboard. Acara 27 mengintegrasikan backend dan frontend secara lebih utuh: operasi *Create*, *Update*, dan *Delete* dipisah menjadi halaman view tersendiri (form_tambah.html dan form_edit.html) yang terhubung ke MahasiswaController melalui Router, lengkap dengan token autentikasi, notifikasi visual, dan validasi di sisi View.

2. Tujuan

1. Membuat view tambah data (form_tambah.html) yang mengirim data dan foto dengan FormData disertai header Authorization.
2. Membuat view ubah data (form_edit.html) yang memuat data lama berdasarkan id pada query string dan menyimpan perubahannya.
3. Menambahkan tombol hapus pada tabel dashboard yang memperbarui tabel tanpa muat ulang halaman penuh.
4. Menguji alur CRUD penuh dari sisi View: tambah, ubah, dan hapus oleh admin.
5. Menambahkan notifikasi visual untuk setiap operasi Create/Update/Delete, baik berhasil maupun gagal (tugas mandiri 1).
6. Menambahkan validasi sisi View (field wajib diisi) sebelum data dikirim ke Controller (tugas mandiri 2).

3. Dasar Teori

Antarmuka *FormData* menyusun pasangan *key-value*, termasuk berkas, untuk dikirim dengan *fetch()* dalam format *multipart/form-data*. Menurut dokumentasi MDN, header *Content-Type* tidak boleh diatur manual saat mengirim *FormData* karena browser perlu menambahkan *boundary* pemisah antar-field; mengaturnya sendiri membuat server gagal membaca body.
Pada frontend JavaScript murni, operasi *Update* dan *Delete* dilakukan dengan mengirim method melalui opsi *method* pada *fetch()*, disertai id data pada *query string* dan token pada header *Authorization*. Khusus pembaruan yang membawa berkas, PHP hanya mengisi *$_POST* dan *$_FILES* untuk request POST, sehingga method PUT disimulasikan dengan POST dan parameter *_method=PUT* yang dikenali Router (Acara 26). Validasi di sisi View memberi umpan balik cepat kepada pengguna, tetapi tidak menggantikan validasi di server karena request dapat dikirim tanpa melalui halaman web.

HASIL PRAKTIK

1. Implementasi

a. Perubahan File

Acara 27 hanya mengubah lapisan View. Model, Controller, Router, dan basis data tidak berubah dari Acara 26.

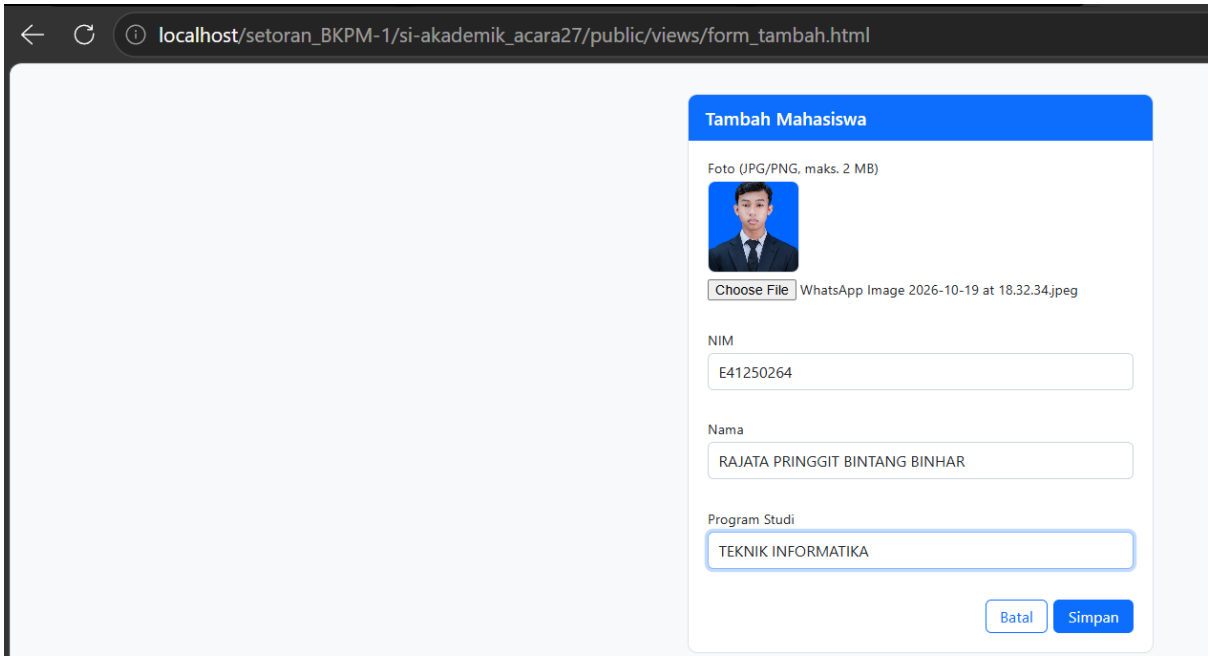
Tabel 1. Daftar perubahan file

File	Status	Keterangan
public/views/form_tambah.html, form_tambah.js	Baru	Form tambah (Create): FormData, header Authorization, pratinjau foto, validasi View
public/views/form_edit.html, form_edit.js	Baru	Form ubah (Update): memuat data lama dari ?id=, simpan dengan POST _method=PUT
public/views/dashboard.html, dashboard.js	Diubah	Tombol Tambah/Ubah menjadi tautan ke halaman form; tombol Hapus memperbarui tabel tanpa reload penuh
public/views/common.js	Diubah	Notifikasi (toast), Validasi, wajibAdmin(), urlFoto()
public/views/style.css	Diubah	Gaya toast, tautan bergaya tombol, halaman form
README.md, docs/TEST_CASE_ACARA27.md	Baru/Diubah	Catatan perubahan dan skenario uji View

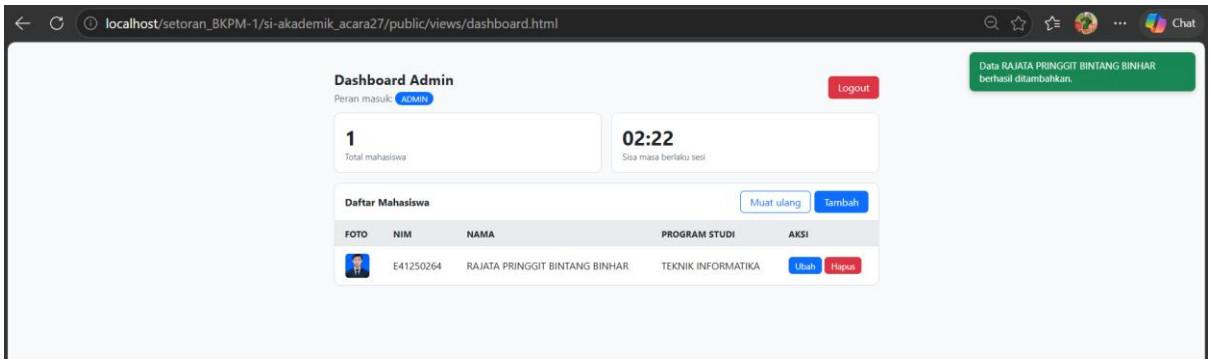
b. Langkah 1 - Membuat View Tambah Data (form_tambah.html)

Form berisi field foto (opsional), NIM, nama, dan program studi. Data dikirim dengan *FormData* melalui fungsi panggilApi() yang menambahkan header *Authorization: Bearer <token>* dan tidak mengatur *Content-Type* secara manual. Foto hanya dilampirkan bila dipilih. Setelah berhasil, pesan sukses disimpan di *sessionStorage* lalu pengguna dialihkan ke dashboard yang menampilkannya sebagai notifikasi.

```
// form_tambah.js (cuplikan)
const formData = new FormData();   // tanpa Content-Type manual
formData.append('nim', data.nim);
formData.append('nama', data.nama);
formData.append('prodi', data.prodi);
if (file) formData.append('foto', file);   // foto opsional
const { ok, status, hasil } = await panggilApi('/mahasiswa',
    { method: 'POST', auth: true, body: formData });
if (ok) {
    Notifikasi.simpanUntukBerikut(`Data ${data.nama} berhasil ditambahkan.`);
    window.location.href = 'dashboard.html';
}
```



Gambar 1. Halaman form tambah mahasiswa



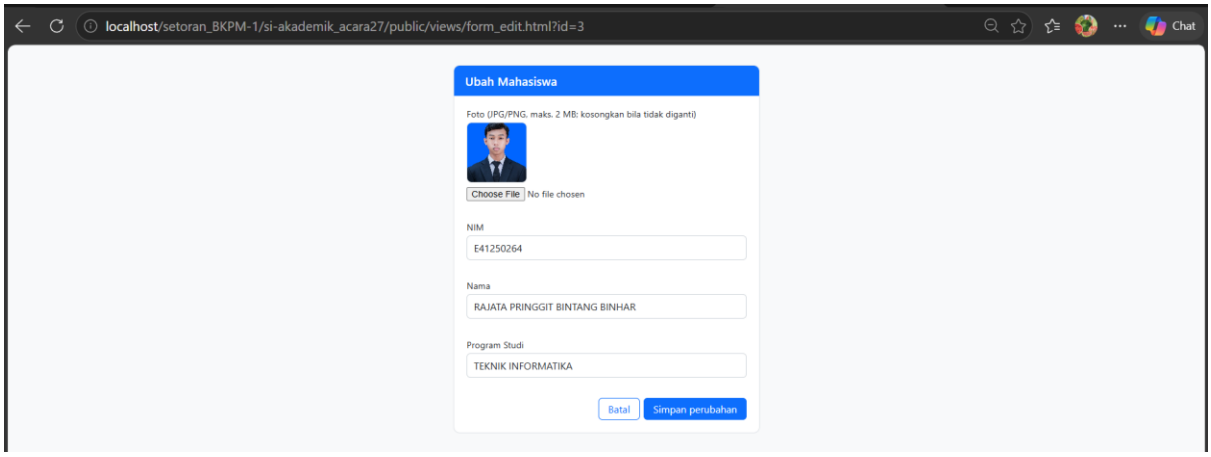
Gambar 2. Dashboard setelah data berhasil ditambahkan

c. Langkah 2 - Membuat View Edit Data (form_edit.html)

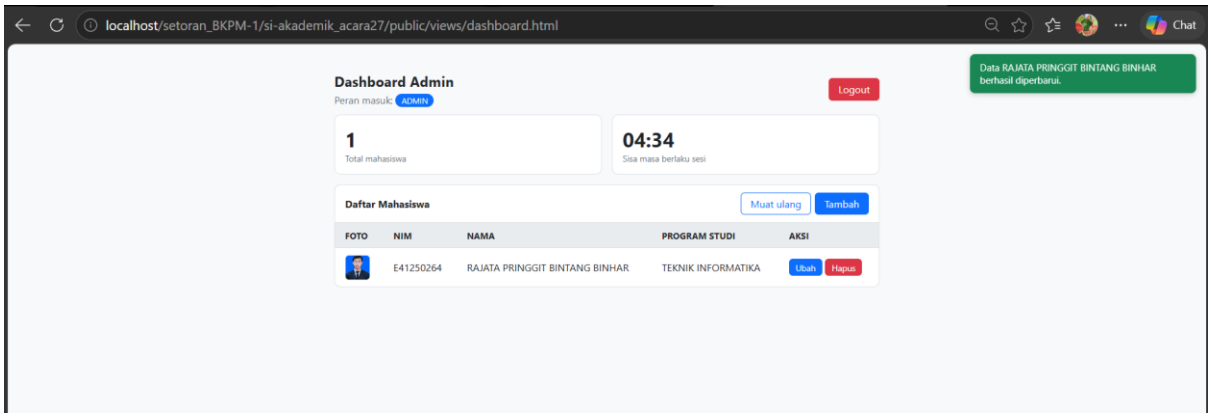
Halaman dibuka dengan alamat form_edit.html?id=N. Data lama diambil dengan *GET /mahasiswa?id=N* (MahasiswaController::index) lalu diisikan ke form, termasuk pratinjau foto lama. Bila id tidak valid atau datanya tidak ada (404), form tidak ditampilkan dan pesan galat muncul. Foto baru bersifat opsional: bila tidak dipilih, server mempertahankan foto lama.

```
// form_edit.js (cuplikan)
const { ok, status, hasil } = await panggilApi(`/mahasiswa?id=${id}`);
            form.elements.nim.value = mhs.nim;
            form.elements.nama.value = mhs.nama;
            form.elements.prodi.value = mhs.prodi;

// PHP tidak mengisi $_POST/$_FILES pada PUT:
// kirim POST + _method=PUT
await panggilApi(`/mahasiswa?id=${id}&_method=PUT`, { method: 'POST', auth:
true, body: formData });
```



Gambar 3. Halaman form ubah mahasiswa



Gambar 4. Dashboard setelah data berhasil diperbarui

Tabel 2 merangkum perbedaan implementasi ini dengan cuplikan modul.

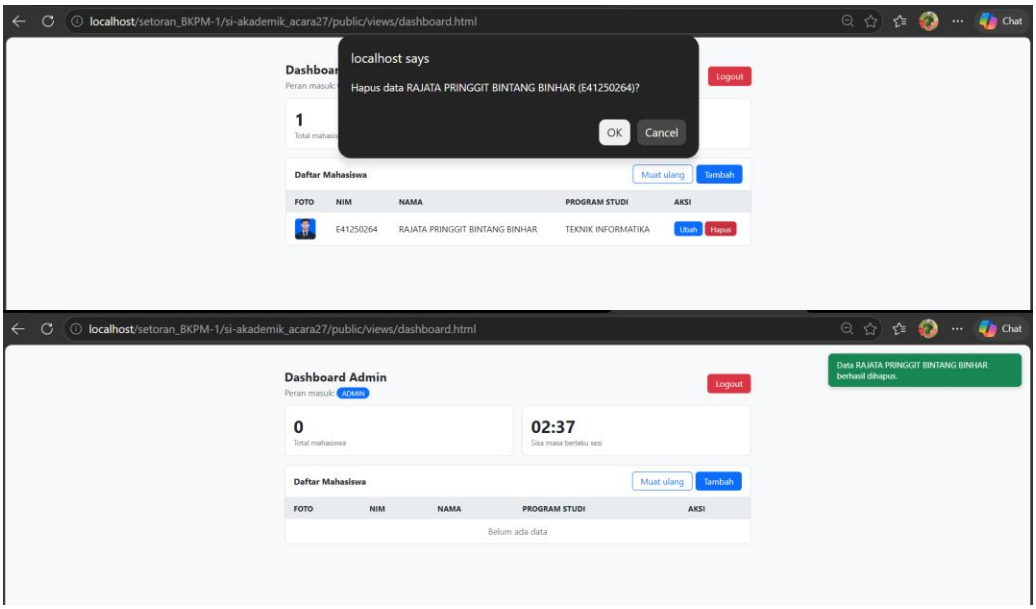
Tabel 2. Perbedaan dengan cuplikan modul

Hal	**Cuplikan modul**	**Implementasi ini**
Method ubah data	fetch() method PUT dengan FormData	POST ?id=..&_method=PUT; PHP tidak mengisi $_POST/$_FILES pada PUT sehingga data kosong dan ditolak 400
Pemanggil API	fetch() polos	panggilApi(): token ditambahkan otomatis; 401 (token habis) kembali ke login
Foto pada form ubah	Tidak ada	Field foto opsional dengan pratinjau; tanpa foto baru, foto lama dipertahankan
Hasil operasi	Tidak dibahas	Toast sukses/gagal dan pesan galat per field dari respons API
Akses non-admin	Tidak dibahas	wajibAdmin(): role user dialihkan ke dashboard; server tetap menolak 403

d. Langkah 3 - Menambahkan Tombol Hapus pada Tabel Dashboard

Setiap baris tabel memiliki tombol Ubah (tautan ke form_edit.html?id=..) dan Hapus. Tombol Hapus meminta konfirmasi, mengirim *DELETE /mahasiswa?id=..* dengan token, lalu memuat ulang isi tabel melalui fungsi muatDataMahasiswa() tanpa memuat ulang halaman. Di server, MahasiswaController::destroy() juga menghapus berkas foto terkait.

```
// dashboard.js (cuplikan)
async function hapusMahasiswa(mhs) {
    if (!confirm(`Hapus data ${mhs.nama} (${mhs.nim})?`)) return;
    const { ok, status, hasil } = await panggilApi(`/mahasiswa?id=${mhs.id}`,
        { method: 'DELETE', auth: true });
    if (!ok) {
        Notifikasi.tampil(`Gagal menghapus data: ${hasil.message}`, 'galat');
        return;
    }
    Notifikasi.tampil(`Data ${mhs.nama} berhasil dihapus.`);
    await muatDataMahasiswa();   // tabel diperbarui tanpa reload
}
```



Gambar 5. Penghapusan data dari dashboard

e. Langkah 4 - Pengujian Alur CRUD Penuh dari Sisi View

Pengujian dilakukan sebagai admin: menambah data baru lengkap dengan foto melalui form_tambah.html, mengubah salah satu data melalui form_edit.html dan memeriksa perubahannya di dashboard, lalu menghapus salah satu data dan memastikan baris hilang dari tabel dan basis data. Hasil lengkap ada pada bagian Pengujian (Tabel 3).

2. Tugas Praktik Mandiri

a. Notifikasi Visual untuk Operasi Create/Update/Delete

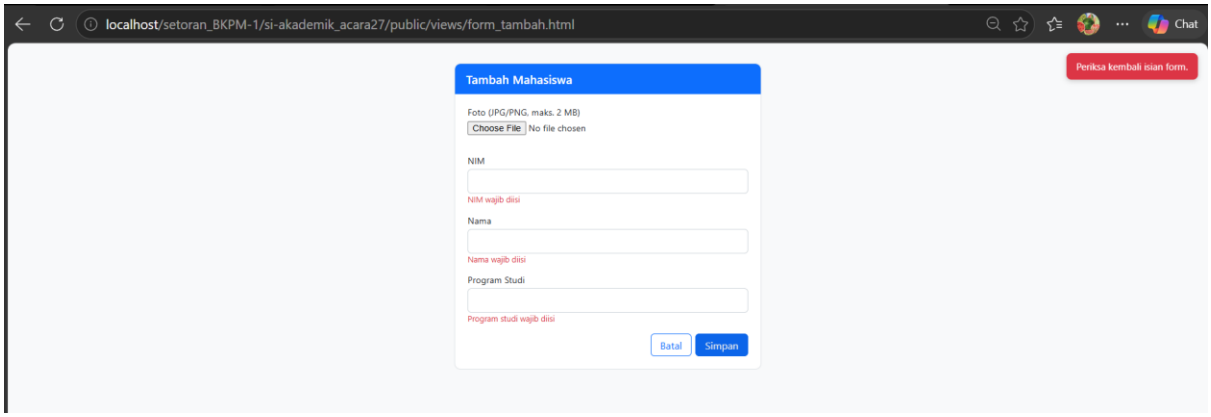
Objek Notifikasi di common.js menampilkan kotak notifikasi (*toast*) di pojok kanan atas yang hilang otomatis setelah 4 detik: hijau untuk berhasil dan merah untuk gagal. Teks dipasang dengan *textContent* sehingga pesan dari server tidak diinterpretasikan sebagai HTML. Karena tambah dan ubah berpindah halaman, pesan disimpan sementara di *sessionStorage* dan ditampilkan oleh dashboard saat dimuat.

```
// common.js (cuplikan)
tampil(teks, jenis = 'ok') {
    const el = document.createElement('div');
    el.className = `toast ${jenis}`;
    el.textContent = teks;
    wadah.appendChild(el);
    setTimeout(() => el.remove(), 4000);
}
```

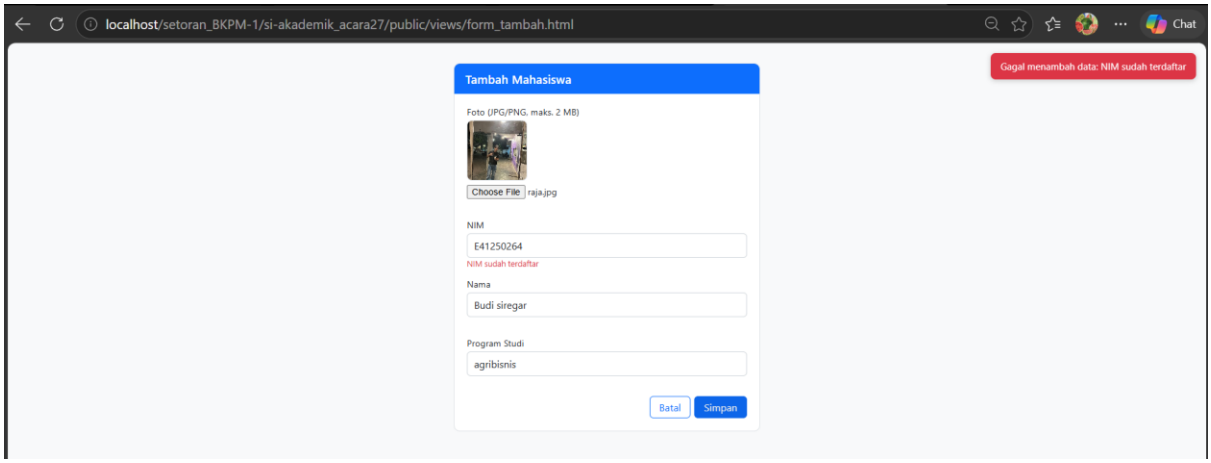
b. Validasi Sisi View

Objek Validasi memeriksa NIM, nama, dan program studi wajib diisi, NIM harus 9 karakter (sama dengan aturan server), serta foto harus JPG/PNG dan tidak lebih dari 2 MB. Bila ada kesalahan, pesan muncul di bawah field terkait, notifikasi merah tampil, dan data tidak dikirim ke server. Validasi ini lapisan tambahan; validasi yang menentukan tetap berada di MahasiswaController.

```
// common.js (cuplikan)
mahasiswa({ nim, nama, prodi }, file = null) {
    const galat = {};
    if (!nim) galat.nim = 'NIM wajib diisi';
    else if (nim.length !== 9) galat.nim = 'NIM harus 9 digit';
    if (!nama) galat.nama = 'Nama wajib diisi';
    if (!prodi) galat.prodi = 'Program studi wajib diisi';
    const galatFoto = this.foto(file);
    if (galatFoto) galat.foto = galatFoto;
    return galat;
}
```



Gambar 6. Validasi sisi View pada form tambah



Gambar 7. Notifikasi kegagalan dari respons API

3. Pengujian

Pengujian dilakukan pada dua lapisan. Pertama, logika View diuji otomatis dengan jsdom menggunakan respons API tiruan: 17 skenario mencakup validasi field kosong tanpa request ke server, pengiriman *FormData* tanpa *Content-Type* manual dengan header *Authorization*, penolakan role user, pemuatan data pada form ubah, pengiriman *POST ?id=..&_method=PUT*, tautan tombol Ubah, serta penghapusan dengan notifikasi dan pembaruan tabel; seluruhnya lulus. Kedua, alur CRUD diuji manual di browser terhadap Apache dan MySQL (Tabel 3).

Tabel 3. Hasil pengujian alur CRUD dari sisi View

ID	Skenario	Hasil yang diharapkan	Hasil
V-01	Tambah data valid dengan foto	Toast hijau; baris baru berfoto tampil di dashboard	[isi]
V-02	Tambah data tanpa foto	Data tersimpan; kolom foto bertanda "-"	[isi]
V-03	Tambah dengan semua field kosong	Pesan "wajib diisi" per field; tidak ada request ke server	[isi]
V-04	Tambah dengan NIM bukan 9 karakter	Pesan "NIM harus 9 digit"; tidak ada request ke server	[isi]
V-05	Tambah dengan foto bukan gambar atau lebih dari 2 MB	Pesan galat pada field foto; Simpan ditolak	[isi]
V-06	Tambah dengan NIM yang sudah ada	Toast merah; pesan 409 pada field NIM	[isi]
V-07	Ubah data tanpa mengganti foto	Toast hijau; data berubah; foto lama tetap	[isi]
V-08	Ubah data dengan foto baru	Foto berganti; berkas lama terhapus dari public/uploads	[isi]
V-09	Buka form_edit.html?id=9999	Pesan "Data tidak ditemukan"; form tidak tampil	[isi]
V-10	Hapus data lalu konfirmasi OK	Toast hijau; baris hilang tanpa reload penuh; foto terhapus	[isi]
V-11	Hapus data lalu batal (Cancel)	Tidak ada perubahan	[isi]
V-12	Buka form_tambah.html sebagai role user	Dialihkan ke dashboard dengan notifikasi akses ditolak	[isi]
V-13	Simpan setelah masa berlaku token habis	Kembali ke login.html?sesi=habis	[isi]

4. Pembahasan

Pemisahan tambah dan ubah menjadi halaman tersendiri membuat setiap view memiliki satu tanggung jawab, sementara dashboard fokus menampilkan daftar dan aksi hapus. Seluruh permintaan melewati Router yang sama dan penegakan hak akses tetap dilakukan server melalui Auth::requireAdmin(); fungsi wajibAdmin() di View hanya memberi kenyamanan dan tidak boleh dianggap sebagai pengamanan.

Temuan utama adalah cuplikan modul untuk form ubah memakai method PUT dengan *FormData*. PHP tidak mengisi *$_POST* dan *$_FILES* untuk request PUT, sehingga data tiba kosong dan ditolak dengan kode 400. Solusinya sama dengan Acara 26: mengirim POST dengan *_method=PUT* yang hanya diakui Router untuk PUT. Selain itu, *Content-Type* tidak diatur manual agar browser menambahkan *boundary*.

Notifikasi dan validasi View meningkatkan pengalaman pengguna, tetapi keduanya tidak menggantikan validasi server: skenario NIM duplikat (409) hanya dapat dideteksi di server dan ditampilkan kembali sebagai pesan pada field NIM. Keterbatasan: pesan antarhalaman disimpan di sessionStorage sehingga hanya tampil sekali pada halaman tujuan; token tersimpan di localStorage dan dapat dibaca oleh skrip pada origin yang sama; serta penyuntingan dan penghapusan belum memiliki fitur urung (*undo*).

KESIMPULAN

1. View tambah (form_tambah.html) dan ubah (form_edit.html) terintegrasi dengan MahasiswaController melalui Router dengan FormData dan token autentikasi.
2. Pembaruan data yang membawa foto harus dikirim sebagai POST dengan _method=PUT karena PHP tidak memproses multipart pada PUT.
3. Tombol hapus pada dashboard menghapus data dan foto serta memperbarui tabel tanpa memuat ulang halaman.
4. Notifikasi visual dan validasi sisi View memberi umpan balik yang jelas kepada pengguna, dengan validasi server tetap menjadi penentu.
5. Alur CRUD penuh dari sisi View berjalan sesuai skenario pengujian pada Tabel 3.

DAFTAR PUSTAKA

Pratama, R. A., & Hasanah, Q. (2026). *Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805)*. Program Studi D4 Teknik Informatika PSDKU Nganjuk, Jurusan Teknologi Informasi, Politeknik Negeri Jember.

MDN Web Docs. (n.d.). *Using FormData objects*. Mozilla.